Introduction to Logical Operators and Comments

1. And Operator (&&)

The AND operator (`&&`) returns `true` only when both conditions are true. If either condition is false, the result is false.

Truth Table for AND Operator

Condition 1	Condition 2	Result
true	true	true
true	false	false
false	true	false
false	false	false

Syntax

```java
condition1 && condition2
```

Examples

```java
public class AndOperatorExample {
    public static void main(String[] args) {
        int age = 20;
        boolean hasLicense = true;

        // Both conditions must be true
        if (age >= 18 && hasLicense) {
            System.out.println("You can drive!");
        }

        // Example with numbers
        int x = 5;
        int y = 10;
        if (x > 0 && y > 0) {
            System.out.println("Both numbers are positive");
        }

        // Example that evaluates to false
        if (x > 10 && y > 5) {
            System.out.println("This won't print");
        } else {
            System.out.println("At least one condition is false");
        }
    }
}
```

2. Or Operator (||)

The OR operator (`||`) returns `true` when at least one condition is true. It only returns false when both conditions are false.

Truth Table for OR Operator

Condition 1	Condition 2	Result
true	true	true
true	false	true
false	true	true
false	false	false

Syntax

```java
condition1 || condition2
```

Examples

```java
public class OrOperatorExample {
    public static void main(String[] args) {
        int temperature = 35;
        boolean isWeekend = false;

        // At least one condition must be true
        if (temperature > 30 || isWeekend) {
            System.out.println("Perfect day for swimming!");
        }

        // Example with user input validation
        char grade = 'B';
        if (grade == 'A' || grade == 'B' || grade == 'C') {
            System.out.println("Passing grade!");
        }

        // Example that evaluates to false
        int score = 45;
        if (score > 90 || score < 0) {
            System.out.println("This won't print");
        } else {
            System.out.println("Score is between 0 and 90");
        }
    }
}
```

3. Not Operator (!)

The NOT operator (`!`) reverses the boolean value. It converts `true` to `false` and `false` to `true`.

Truth Table for NOT Operator

Condition	Result
true	false
false	true

Syntax

```java
!condition
```

Examples

```java
public class NotOperatorExample {
    public static void main(String[] args) {
        boolean isRaining = false;

        // Using NOT operator
        if (!isRaining) {
            System.out.println("Great day for a picnic!");
        }

        // NOT with comparison operators
        int number = 5;
        if (!(number > 10)) {
            System.out.println("Number is not greater than 10");
        }

        // NOT with other logical operators
        boolean hasTicket = true;
        boolean isVIP = false;

        if (hasTicket && !isVIP) {
            System.out.println("Regular admission");
        }

        // Double NOT (returns original value)
        boolean value = true;
        System.out.println("Original: " + value);
        System.out.println("Single NOT: " + !value);
        System.out.println("Double NOT: " + !!value);
    }
}
```

Combining Logical Operators

```java
public class CombinedLogicalOperators {
    public static void main(String[] args) {
        int age = 25;
        boolean hasJob = true;
        boolean hasGoodCredit = true;
        boolean isStudent = false;

        // Complex condition using multiple operators
        if ((age >= 18 && hasJob) || (isStudent && !hasGoodCredit)) {
            System.out.println("Eligible for loan");
        }

        // Using parentheses for clarity
        if (!(age < 18) && (hasJob || isStudent)) {
            System.out.println("Can apply for credit card");
        }
    }
}
```

4. Adding Comments in the Code

Comments are text in your code that are ignored by the compiler. They help explain what your code does and make it easier to understand for yourself and others.

Types of Comments in Java

Single-line Comments (`//`)

Used for short comments on a single line.

```java
// This is a single-line comment
int age = 20; // You can also add comments at the end of a line
```

Multi-line Comments (`/* */`)

Used for longer comments that span multiple lines.

```java
/*
This is a multi-line comment.
It can span several lines.
Useful for longer explanations.
*/
int total = 0;
```

Documentation Comments (`/** */`)

Special comments used to generate documentation. They start with `/**` and end with `*/`.

```java
/**
 * This method calculates the area of a rectangle
 * @param length the length of the rectangle
 * @param width the width of the rectangle
 * @return the area of the rectangle
 */
public static int calculateArea(int length, int width) {
    return length * width;
}
```

Examples of Good Comments

```java
public class CommentExample {
    public static void main(String[] args) {
        // Declare variables for student information
        String studentName = "John";
        int studentAge = 20;
        double gpa = 3.75;

        /*
         * Check if student is eligible for scholarship
         * Requirements: age >= 18 and GPA >= 3.5
         */
        if (studentAge >= 18 && gpa >= 3.5) {
            System.out.println(studentName + " is eligible for scholarship");
        }

        // Calculate grade based on GPA
        char grade;
        if (gpa >= 3.7) {
            grade = 'A'; // Excellent performance
        } else if (gpa >= 3.0) {
            grade = 'B'; // Good performance
        } else {
            grade = 'C'; // Average performance
        }

        System.out.println("Grade: " + grade);
    }
}
```

Best Practices for Comments

- **Be Clear and Concise**: Write comments that are easy to understand
- **Explain Why, Not What**: Focus on explaining the purpose rather than obvious actions
- **Keep Comments Updated**: Update comments when you change the code
- **Don't Over-Comment**: Avoid commenting obvious code

```java
// Good comment
// Calculate tax amount (8.5% sales tax)
double tax = price * 0.085;

// Poor comment
// Multiply price by 0.085
double tax = price * 0.085;
```

Quiz Questions

1. What will be the result of the expression `true && false`? a) true b) false c) null d) error

Answer: b) false

2. Which logical operator returns true when at least one condition is true? a) && (AND) b) || (OR) c) ! (NOT) d) == (EQUALS)

Answer: b) || (OR)

3. What does the expression `!(5 > 3)` evaluate to? a) true b) false c) 5 d) 3

Answer: b) false

4. Which type of comment is used for documentation generation in Java? a) // single-line comment b) /* multi-line comment */ c) /* documentation comment */ d) # hash comment

Answer: c) / documentation comment /*

5. What will be the output of `System.out.println(true || false && false);`? a) true b) false c) error d) null

Answer: a) true (because && has higher precedence than ||, so it evaluates as `true || (false && false)` which is `true || false` = `true`)

Practice Coding Questions

1. Write a Java program that checks if a person is eligible to vote. A person can vote if they are 18 years or older AND they are a citizen. Use logical operators to implement this condition.

2. Create a program that determines if a student gets a discount at a movie theater. Students get a discount if they are under 25 OR they have a student ID. Use the OR operator to check this condition.

3. Write a program that checks if a number is NOT between 10 and 20 (exclusive). Use the NOT operator in your solution.

4. Create a program that determines if someone can get a loan. The conditions are: (age >= 21 AND has a job) OR (has collateral AND income > 50000). Use parentheses and logical operators to implement this complex condition.

5. Write a well-commented Java program that calculates the final grade for a student based on three test scores. Include single-line comments explaining each step and a multi-line comment explaining the grading scale at the top of your program.